- Placify — Project Abstract & Documentation
  - 📌 Abstract
  - 📖 Description
  - ❓ Problem Statement — What Problem Does Placify Solve?
  - 🏗️ System Architecture — Expected Overall Flow
    - Request Lifecycle
  - 🛠️ Tech Stack & Dependencies
    - Frontend
    - Backend
    - Database
  - 🤖 AI Stack
    - Model
    - ATS System Prompt Design
    - Link Verification Agent
  - ✅ Features Implemented
    - Authentication & User Management
    - Landing Page & UI
    - Dashboard
    - Community Forum
    - ATS Resume Analyzer
    - Backend Infrastructure
  - 🚧 Features To Be Implemented
    - Resume Builder
    - Chat / Messaging
    - Placed Resumes Showcase
    - AI Enhancements
    - Community & Social
    - Platform & DevOps
  - 🏁 Conclusion

Placify — Project Abstract & Documentation

> **Course:** Full Stack Development (Semester 4)
> **Institution:** Vellore Institute of Technology (VIT)
> **Project Type:** Full-Stack Web Application with AI Integration

📌 Abstract

Placify is an AI-augmented placement preparation and career networking platform designed for students and working professionals. It centralises the entire placement journey — from community-driven knowledge sharing and peer networking to intelligent resume analysis — into a single cohesive web application. By integrating a Large Language Model (LLM) powered ATS (Applicant Tracking System) analyzer with real-time link verification, Placify bridges the gap between raw placement guidance and actionable, personalised career insights.

📖 Description

Placify is a role-aware, full-stack web application that serves three distinct user personas:

- **Students (Unplaced):** Seeking guidance, resume help, job opportunities, and community support.
- **Students (Placed):** Sharing experiences, strategies, and sample resumes with peers.
- **Working Professionals (HR/Employee):** Posting opportunities, engaging with the community, and offering mentorship.

After registration, users complete a personalised onboarding flow that captures their profile type and role, which governs the features and content they see. The platform combines a real-time community forum, an AI-powered ATS resume scanner, and a resume builder module — all behind a secure, JWT-authenticated backend.

❓ Problem Statement — What Problem Does Placify Solve?

The campus placement process is fragmented and opaque for most students:

Problem	Impact
Lack of centralised placement insights	Students rely on scattered WhatsApp groups and word-of-mouth
No feedback on resume quality before applying	Resumes get silently rejected by ATS systems
Broken/fabricated links on resumes go undetected	Recruiters distrust candidate profiles
Placed students have no structured channel to give back	Knowledge transfer is lost each batch
HR/recruiters have no direct channel to college communities	Job postings don't reach the right audience

Placify solves this by providing:

1. A **structured community forum** categorised by placement topics.
2. An **AI-powered ATS analyzer** that scores resumes, flags formatting issues, and suggests improvements.
3. A **link verification engine** that independently validates every URL in a resume (GitHub repos, LinkedIn, certifications) before the AI evaluates it.
4. A **placed-students repository** where verified past candidates can share winning resumes and strategies.

🏗️ System Architecture — Expected Overall Flow

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          CLIENT (React SPA)                              │
│                                                                          │
│   Landing Page → Register/Login → Onboarding → Dashboard                 │
│                                                                          │
│   Dashboard Sections:                                                    │
│   ┌──────────────┐  ┌────────────────┐  ┌──────────────────┐  ┌──────────┐ │
│   │  Community   │  │ ATS Analyzer   │  │ Resume Builder   │  │  Chat    │ │
│   │  (Forum)     │  │  (AI + Link    │  │  (Stub/WIP)      │  │  (WIP)   │ │
│   │              │  │  Verifier)     │  │                  │  │          │ │
│   └──────────────┘  └────────────────┘  └──────────────────┘  └──────────┘ │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
```

```
                          │ REST API (CORS-enabled JSON)
                          ▼
┌──────────────────────────────────────────────────────────────────────┐
│                   BACKEND (Express.js — MVC Pattern)                 │
│                                                                      │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────────────────┐ │
│  │  Auth Layer  │   │  Posts/Forum │   │       ATS Module         │ │
│  │              │   │              │   │                          │ │
│  │ - Register   │   │ - CRUD Posts │   │ 1. Upload resume (PDF/DOCX) │ │
│  │ - Login      │   │ - Like/Unlike│   │ 2. Extract text          │ │
│  │ - Onboarding│   │ - Comments   │   │ 3. Run Link Verifier     │ │
│  │ - JWT Auth   │   │ - Pagination │   │    ├─ Plain-text URL regex │ │
│  │ - Profile    │   │ - Categories │   │    ├─ PDF annotation (/URI) │ │
│  └──────────────┘   └──────────────┘   │    └─ DOCX .rels XML     │ │
│         │                 │            │ 4. HTTP HEAD / GitHub API │ │
│         ▼                 ▼            │    check each link       │ │
│  ┌──────────────────────────────┐      │ 5. Send resume + JD + link │ │
│  │  JWT Middleware (Auth Guard) │      │    results → Groq LLM    │ │
│  └──────────────────────────────┘      │ 6. Return structured JSON │ │
│                                        └──────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
                          │ Mongoose ODM
                          ▼
              ┌──────────────────────────┐
              │   MongoDB (Atlas / Local) │
              │                          │
              │  Collections:            │
              │  ├─ Users                │
              │  ├─ Posts                │
              │  └─ Comments             │
              └──────────────────────────┘
                          │
                 External APIs
                 ├─ Groq Cloud (LLM inference)
                 └─ GitHub REST API v3 (repo metadata)
```

Request Lifecycle

1. **User visits** → React SPA served (Create React App)
2. **Auth** → Register/Login → JWT issued → stored in `localStorage`
3. **Onboarding** → Profile type + role captured → persisted to MongoDB
4. **Dashboard** → Role-aware sidebar renders relevant sections
5. **Community** → REST calls to `/api/posts` (paginated, filterable by category)
6. **ATS Analyzer** → `multipart/form-data` POST to `/api/ats/analyze`:
   - Text extracted from PDF/DOCX
   - All URLs extracted (text + embedded hyperlinks)
   - Each URL verified (HTTP HEAD or GitHub API)
   - Groq LLM called with resume text + JD + link verification summary

- Structured JSON response returned and rendered in dashboard panels

---

🛠️ Tech Stack & Dependencies

Frontend

Category	Technology	Version
UI Framework	React	18.2.0
Routing	React Router DOM	6.30.3
Styling	Tailwind CSS	3.4.19
Icons	Lucide React	0.575.0 (devDep)
Build Tool	Create React App (react-scripts)	5.0.1
PostCSS	Autoprefixer	10.4.27

Backend

Category	Technology	Version
Runtime	Node.js	LTS
Web Framework	Express	5.2.1
Database ODM	Mongoose	8.18.0
Authentication	JSON Web Token (jsonwebtoken)	9.0.2
Password Hashing	bcryptjs	2.4.3
File Uploads	Multer	2.1.1
PDF Parsing	pdf-parse	1.1.1
DOCX Parsing	Mammoth	1.11.0
ZIP Handling (DOCX)	JSZip	3.10.1
Cross-Origin	cors	2.8.6
Environment Config	dotenv	16.4.5

Category	Technology	Version
AI SDK	groq-sdk	0.37.0
Dev Server	nodemon	3.1.7

Database

Technology	Role
MongoDB	Primary NoSQL document database
Mongoose	Schema definition, validation, ODM

🤖 AI Stack

Model

Component	Details
Provider	Groq Cloud — ultra-fast LLM inference
Model	`llama-3.3-70b-versatile` (70B parameter Meta LLaMA 3.3)
Temperature	0.2 (deterministic, low creativity for structured output)
Max Tokens	3,000 per analysis response

ATS System Prompt Design

The AI is given a carefully engineered system prompt that instructs it to:

- Evaluate the resume against ATS criteria using a **7-section rubric** (header, summary, experience, education, skills, certifications, projects)
- Perform **keyword matching** against the provided job description (or infer the role if none is provided)
- Detect **formatting issues** (tables, graphics, columns, non-standard fonts) that break ATS parsers

- Return a **strict JSON-only output** (no markdown preamble) with fields: `score`, `scoreRationale`, `detectedRole`, `sectionNotes`, `keywordMatch`, `formattingIssues`, `improvementChecklist`, `nextSteps`

Link Verification Agent

A custom rule-based verification agent (`linkVerifier.js`) runs **before** the LLM call:

Method	Description
Plain-text regex	Extracts all `http(s)://` URLs from the raw resume text
PDF binary scan	Parses `/URI` annotation entries directly from the PDF byte stream (3 patterns: parenthesis, hex-encoded, `/A <<` style)
DOCX relationship XML	Unzips the DOCX archive and parses all `word/**/*.rels` XML files for `Type="…/hyperlink"` entries
GitHub API check	Calls `api.github.com/repos/{owner}/{repo}` to get live metadata (stars, language, last push date, visibility)
HTTP HEAD check	Fires an HTTP HEAD request for all other URLs; treats HTTP 2xx–3xx and 403/405 (bot-blocked live servers) as `live`

Verification results are formatted and injected into the Groq prompt, allowing the LLM to factor dead links into the score and flag them in the improvement checklist.

✅ Features Implemented

Authentication & User Management

- ☑ User registration with email + password (bcrypt hashed)
- ☑ JWT-based login with 7-day token expiry
- ☑ Protected routes via `authenticate` middleware
- ☑ Role-aware onboarding (Student/Placed/Unplaced, Working Professional/HR/Employee)

- ☑ LinkedIn profile URL on user profile
- ☑ User search / directory (`GET /api/auth/users?search=`)

Landing Page & UI

- ☑ Full marketing landing page (Hero, Problem Section, How It Works, For Senior Section, Social Proof, CTA, Footer)
- ☑ Responsive Navbar with auth state awareness
- ☑ Smooth scroll, animated components

Dashboard

- ☑ Role-aware sidebar navigation
- ☑ Dashboard home with personalised stats and quick-access cards
- ☑ User profile section (view & update LinkedIn URL)

Community Forum

- ☑ Create, read, and delete posts
- ☑ Category filtering (All Posts, Internships, Placements, DSA, Aptitude, Interview Experience, etc.)
- ☑ Paginated post feed (20 posts per page)
- ☑ Like / Unlike posts (toggle)
- ☑ Nested comments (add & delete own comments)
- ☑ Comment count on post cards

ATS Resume Analyzer

- ☑ Resume upload (PDF and DOCX supported)
- ☑ Optional job description input for targeted analysis
- ☑ AI-powered ATS scoring (0–100) with detailed rationale
- ☑ Section-by-section analysis (header, summary, experience, education, skills, certifications, projects)
- ☑ Keyword match report (matched, partial, missing, density)
- ☑ Formatting issue detection (severity: high/medium/low)
- ☑ Prioritised improvement checklist with before/after examples

- ☑ Next steps summary
- ☑ Link verification panel (live/dead status for GitHub repos, LinkedIn, and certifications)
- ☑ GitHub repository metadata display (stars, language, last push, visibility)

Backend Infrastructure

- ☑ MVC-structured Express backend
- ☑ Global error handler middleware (`notFound`, `errorHandler`)
- ☑ CORS policy with configurable `CLIENT_ORIGIN`
- ☑ Health check endpoint (`GET /api/health`)
- ☑ Environment variable management with dotenv

🚧 Features To Be Implemented

Resume Builder

- ☐ Interactive resume builder with section-by-section editor
- ☐ Multiple resume templates (ATS-safe designs)
- ☐ PDF export of the generated resume
- ☐ Auto spell-check and grammar correction

Chat / Messaging

- ☐ Real-time one-on-one chat between users (WebSocket / Socket.IO)
- ☐ Message history persistence in MongoDB
- ☐ Online presence / typing indicators

Placed Resumes Showcase

- ☐ Placed students can upload anonymised winning resumes
- ☐ Category-tagged (Company, Role, Year, Package range)
- ☐ Search and filter placed resume repository
- ☐ Download with access control (login required)

AI Enhancements

- ☐ AI-powered cover letter generator based on resume + JD
- ☐ Streaming ATS analysis output (real-time token-by-token response)
- ☐ Mock interview Q&A generator (role-specific questions from JD)
- ☐ Smart job-description matcher: upload multiple JDs and rank them by resume fit

Community & Social

- ☐ Post bookmarks / save for later
- ☐ Rich text editor for posts (Markdown / WYSIWYG)
- ☐ Upvote/downvote system for comments
- ☐ User follow system and personalised feed
- ☐ Notification system (new comments, likes, mentions)

Platform & DevOps

- ☐ Admin dashboard (user management, post moderation)
- ☐ Email verification on registration
- ☐ Password reset via email (OTP / magic link)
- ☐ Rate limiting and request throttling
- ☐ Deployment pipeline (CI/CD) to production (e.g., Render + Vercel / MongoDB Atlas)
- ☐ Unit and integration tests (Jest + Supertest)

🏁 Conclusion

Placify addresses a real and pressing gap in the campus placement ecosystem by building a unified, intelligent platform that serves students and professionals alike. The current implementation establishes a robust full-stack foundation — a secure JWT-authenticated Express backend following the MVC architectural pattern, a MongoDB data layer with well-structured Mongoose schemas, and a React frontend with Tailwind CSS styling.

The standout feature at this stage is the AI-powered ATS Resume Analyzer, which goes beyond a simple LLM call by incorporating a multi-source link verification agent that independently validates every URL embedded in the resume — from GitHub repositories (fetching live metadata via the GitHub API) to LinkedIn profiles and certification badges — and feeds that verified context into the Groq `llama-3.3-70b-versatile` model for a more accurate, evidence-grounded analysis.

With a clear roadmap of features to implement — real-time chat, resume building, AI cover letter generation, and a placed-students repository — Placify is engineered to grow into a comprehensive career platform that empowers students to enter the workforce confidently and prepared.

Documentation generated: March 2026